

JOBSHEET 15
LAPORAN PERCOBAAN PRAKTIKUM
COLLECTION

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

0. Persiapan

Customer.java

```
public class Customer{
    public int id;
    public String name;

    public Customer() {

    }

    public Customer(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return " ID: " + this.id + " Nama: " + this.name;
    }
}
```

Customer.java

```
public class Book {
    public String isbn;
    public String title;

    public Book() {

    }

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String toString() {
        return "ISBN: " + this.isbn + " Title: " + this.title;
    }
}
```

1. Percobaan 1 : Implementasi ArrayList

DemoArrayList.java

```
import java.util.ArrayList;

public class DemoArrayList {

    public static void main(String[] args) {
        ArrayList<Customer> customers = new ArrayList<>();

        Customer customer1 = new Customer(1, "Zakia");
        Customer customer2 = new Customer(5, "Budi");

        customers.add(customer1);
        customers.add(customer2);

        customers.add(new Customer(4, "Cica"));

        customers.add(2, new Customer(100, "Rosa"));

        System.out.println(customers.indexOf(customer2));

        Customer customer = customers.get(1);
        System.out.println(customer.name);
        customer.name = "Budi Utomo";

        ArrayList<Customer> newCustomers = new ArrayList<>();
        newCustomers.add(new Customer(201, "Della"));
        newCustomers.add(new Customer(202, "Victor"));
        newCustomers.add(new Customer(203, "Sarah"));

        customers.addAll(newCustomers);

        for (Customer cust : customers) {
            System.out.println(cust.toString());
        }

        System.out.println(customers);
    }
}
```

Verifikasi Hasil Percobaan

```
1
Budi
ID: 1 Nama: Zakia
ID: 5 Nama: Budi Utomo
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
ID: 201 Nama: Della
ID: 202 Nama: Victor
ID: 203 Nama: Sarah
[ ID: 1 Nama: Zakia, ID: 5 Nama: Budi Utomo, ID: 100 Nama: Rosa, ID: 4 Nama: Cica, ID: 201 Nama: Della, ID: 202 Nama: Victor, ID: 203 Nama: Sarah]
```

Pertanyaan

4. Cobalah tambahkan object customer baru ke dalam customers. Apakah object dapat ditambahkan meskipun melebihi kapasitas?

```
ArrayList<Customer> customers = new ArrayList<>(2);

Customer customer1 = new Customer(1, "Zakia");
Customer customer2 = new Customer(5, "Budi");

customers.add(customer1);
customers.add(customer2);

customers.add(new Customer(4, "Cica"));

for (Customer cust : customers) {
    System.out.println(cust.toString());
}
```

5. Compile dan run kode program, di mana object yang baru ditambahkan? Di awal, di tengah, atau di akhir collection?
7. Compile dan run kode program. Index pada ArrayList dimulai dari 0 atau 1?
10. Cobalah hapus angka 2 saat instansiasi object customers. Apakah ArrayList dapat diinstansiasi tanpa harus menentukan size di awal?

Jawaban

4. Object dapat ditambahkan meskipun melebihi kapasitas.

```
ID: 1 Nama: Zakia
ID: 5 Nama: Budi
ID: 4 Nama: Cica
```

5. Di akhir collection.

7. Dimulai dari 0, karena index 2 berada pada urutan ketiga.

```
ID: 1 Nama: Zakia
ID: 5 Nama: Budi
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
```

10. Ya, ArrayList dapat diinstansiasi tanpa harus menentukan size di awal.

```
1
Budi
ID: 1 Nama: Zakia
ID: 5 Nama: Budi Utomo
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
```

2. Percobaan 2 : Implementasi Stack

StackDemo.java

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Book book1 = new Book("1234", "Dasar Pemrograman");
        Book book2 = new Book("7145", "Hafalan Shalat Delisa");
        Book book3 = new Book("3562", "Muhammad Al-Fatih");

        Stack<Book> books = new Stack<>();
        books.push(book1);
        books.push(book2);
        books.push(book3);

        Book temp = books.peek();

        if (temp != null) {
            System.out.println(temp.toString());
        }

        Book temp2 = books.pop();

        if (temp2 != null) {
            System.out.println(temp2.toString());
        }

        for (Book book : books) {
            System.out.println(book.toString());
        }

        System.out.println(books);
    }
}
```

Verifikasi Hasil Percobaan

```
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 1234 Title: Dasar Pemrograman
ISBN: 7145 Title: Hafalan Shalat Delisa
[ISBN: 1234 Title: Dasar Pemrograman, ISBN: 7145 Title: Hafalan Shalat Delisa]
```

Pertanyaan

5. Mengapa perlu ada pengecekan (temp != null)?
8. Bagaimana cara melakukan pencarian elemen pada stack menggunakan method search()?

Jawaban

5. Karena ada kemungkinan untuk object tersebut bernilai null yang mana tidak perlu ditampilkan.

8. Cara melakukan pencarian elemen pada Stack adalah dengan menggunakan method search(). Jika yang ditanyakan adalah mekanismenya, maka mekanismenya adalah sebagai berikut. Pertama, program akan menginisialisasi objek yang ada di puncak (top) bernilai 1, objek dibawahnya bernilai 2, dan seterusnya. Method kemudian akan menelusuri stack ke bawah mulai dari elemen paling atas menuju dasar menggunakan method equals() untuk mencocokkan object. Terakhir, method akan mengembalikan indeks lokasi kemunculan pertama dari puncak jika ditemukan ada yang cocok atau -1 jika tidak ditemukan yang cocok.

3. Percobaan 3 : Implementasi TreeSet

TreeSetDemo.java

```
package Pertemuan14; import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<String> fruits = new TreeSet<>();

        fruits.add("Mangga");
        fruits.add("Apel");
        fruits.add("Jeruk");
        fruits.add("Jambu");

        for (String temp : fruits) {
            System.out.println(temp);
        }

        System.out.println("First: " + fruits.first());
        System.out.println("Last: " + fruits.last());

        fruits.remove("Jeruk");
        System.out.println("Setelah remove " + fruits);

        fruits.pollFirst();
        System.out.println("Setelah poll first " + fruits);

        fruits.pollLast();
        System.out.println("Setelah poll last " + fruits);
    }
}
```

Verifikasi Hasil Percobaan

```
Apel  
Jambu  
Jeruk  
Mangga  
First: Apel  
Last: Mangga  
Setelah remove [Apel, Jambu, Mangga]  
Setelah poll first [Jambu, Mangga]  
Setelah poll last [Jambu]
```

Pertanyaan

4. Compile dan run program. Mengapa urutan yang ditampilkan berbeda dengan urutan penambahan data ke dalam TreeSet fruits?
6. Apa yang dilakukan oleh method first(), last(), remove(), pollFirst(), dan pollLast()?

Jawaban

4. Karena ketika menampilkan TreeSet, urutan menampilkan didasarkan pada besar atau kecilnya suatu elemen, elem yang lebih kecil akan ditampilkan terlebih dahulu, barulah elemen yang lebih besar. Pada kasus elemen bertipe data String, program akan melihat dari segi leksikografisnya (urutan secara alfabetis), misalnya “Apel” lebih kecil dari “Jeruk”, sehingga Apel lebih dahulu ditampilkan dibandingkan dengan Jeruk (‘A’ < ‘J’).

6. Method first() mengembalikan elemen pertama (terendah) pada suatu TreeSet.

Method last() mengembalikan elemen terakhir (tertinggi) pada suatu TreeSet.

Method remove() menghapus suatu elemen spesifik.

Method pollFirst() mengambil dan menghapus elemen pertama (terendah) atau mengembalikan null jika TreeSet-nya kosong pada suatu TreeSet.

Method pollLast() mengambil dan menghapus elemen terakhir (tertinggi) atau mengembalikan null jika TreeSet-nya kosong pada suatu TreeSet.

4. Percobaan 4 : Implementasi Sorting

DemoSorting.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;

public class DemoSorting {
    public static void main(String[] args) {
        ArrayList<String> daftarSiswa = new ArrayList<>();

        daftarSiswa.add("Zainab");
        daftarSiswa.add("Andi");
        daftarSiswa.add("Rara");

        Collections.sort(daftarSiswa);

        System.out.println(daftarSiswa);

        // TAMBAHAN
        // -----
        ArrayList<Customer> customers = new ArrayList<>();

        Customer cust1 = new Customer(10, "Neji");
        Customer cust2 = new Customer(13, "Kisame");

        customers.add(cust1);
        customers.add(cust2);
        // -----

        customers.sort((c1, c2) -> c1.name.compareTo(c2.name));

        System.out.println(customers);
    }
}
```

Verifikasi Hasil Percobaan

```
[Andi, Rara, Zainab]
[ ID: 13 Nama: Kisame, ID: 10 Nama: Neji]
```